

ASL Captioning from Video Using Hand Skeleton Tracking

Gabriel Gomes Ferreira,
Nichols O Miles,
Tran Kenny

Team 17

May 1, 2026

Final Project Writeup
HCI 575: Computational Perception

Iowa State University
Spring 2026

ABSTRACT

This project investigates whether American Sign Language (ASL) fingerspelling can be converted into text using hand skeleton tracking and lightweight machine learning. Our original proposal emphasized pre-recorded MP4 transcription and less constrained settings than fixed-box webcam demos. During implementation, we built and validated the core perception-computation pipeline using MediaPipe Hands for 21-point landmark extraction, geometric feature engineering, classical machine learning classifiers, and temporal aggregation for live transcription. The final system supports 24 static fingerspelled letters (A-Y excluding J and Z), and successfully performs both real-time webcam inference and offline transcription of pre-recorded MP4 video files. For both input methods, the system overlays current predictions and an accumulated transcription directly onto the video stream. On a held-out image-level test split derived from the ASL Alphabet dataset, our best classifier, an RBF-kernel support vector machine, achieved 99.28% accuracy, slightly outperforming a random forest baseline at 99.27%. These results show that normalized landmark geometry is highly effective for isolated static-letter recognition. The remaining challenge is not per-frame classification, but robust segmentation and aggregation across continuous signing and more realistic video conditions.

1. Introduction

Accessibility tools have significantly improved for spoken language through automatic speech recognition and captioning systems. However, equivalent tools for sign language remain limited, especially for pre-recorded video content. Currently, if a creator records a video using ASL, captions must be added manually, which is time consuming and requires fluency in both ASL and written English. Existing open-source solutions for sign language recognition primarily focus on live webcam input and require the user to keep their hand within a fixed bounding box. While this approach may work in controlled demonstrations, it is impractical for real world video content where users naturally move their hands across the frame. Our solution's ability to transcribe ASL without a bounding box will allow much more freedom for content creators making ASL videos.

To bridge the gap between constrained laboratory demonstrations and real-world application, this project proposes the development of a dynamic, video-based American Sign Language (ASL) fingerspelling transcription system.

Unlike existing open-source solutions that force users to keep their hands within a rigid, static webcam bounding box, our system processes pre-recorded video files to track natural, unconstrained hand movements and convert gestures into text. By shifting from static processing to dynamic tracking, this project explores a highly practical approach to automated, "in-the-wild" sign language captioning. This video-first architecture is specifically designed to address the unmet needs of several key user populations:

- **Deaf and Hard of Hearing Content Creators:** These individuals frequently share recorded content on platforms like YouTube and social media. Currently, providing English captions for ASL videos requires time-consuming, manual transcription. An automated, video-based system empowers creators to seamlessly reach broader, non-ASL-speaking audiences without the friction of manual data entry.
- **ASL Educators and Students:** Asynchronous language learning relies heavily on prerecorded video. Educators can utilize this system to automatically generate transcripts for instructional materials. Conversely, students recording their fingerspelling homework can use the transcription output as an automated feedback mechanism to verify the clarity and accuracy of their signs before submitting assignments.
- **Accessibility Developers and Archivists:** There is a vast amount of historical, institutional, and archival video footage featuring ASL that currently lacks English transcripts. Because this system processes standard MP4 files rather than requiring a live webcam feed, developers and archivists can leverage it to bulk-analyze existing videos, making older, uncaptioned content searchable, indexable, and accessible.

Our main research question is whether a compact, interpretable pipeline built on hand landmarks and classical machine learning can recognize ASL fingerspelling accurately enough to be useful without requiring a fixed bounding box. A second question is where the real bottleneck lies: isolated letter recognition or temporal segmentation across continuous signing.

2. Related Work

Sign language recognition has been widely explored in both academic research and open-source communities using computer vision and machine learning techniques. Early approaches primarily focused on end-to-end static image classification using Convolutional Neural Networks (CNNs). Public datasets, such as the ASL Alphabet Dataset [1], have made it possible to train models that classify individual alphabet gestures directly from raw pixel data.

While these approaches generally work well in controlled environments, they often rely on fixed camera positioning, are sensitive to lighting and background variations, and require clearly segmented hand regions.

To overcome the limitations of raw pixel variation, many recent systems first estimate hand key points and perform classification based on that structured representation. MediaPipe Hands provides a particularly attractive foundation for this, as it performs real-time 21-point hand landmark extraction from monocular RGB input and is designed for efficient on-device use [2]. Instead of analyzing images, this landmark based method represents the hand using joint coordinates, distances, and angles. Downstream models can then operate on pose geometry, which captures relative finger shape and articulation directly [3]. Recent studies have reported strong recognition performance using classical machine learning classifiers and recurrent networks over these extracted MediaPipe coordinates [4, 5]. This line of work strongly supports our design choice to use normalized hand geometry as the main feature space.

While static frame-level classification is well-studied, dynamic gesture recognition introduces significant complexity. Some research has extended recognition to dynamic gestures in video by incorporating temporal modeling techniques like recurrent 3D CNNs [6]. Furthermore, comprehensive sign language translation systems have been proposed that attempt to translate full ASL sentences by incorporating facial expressions and body pose information [7]. However, these systems require large, annotated datasets and complex architectures, placing them outside the scope of a single-semester project. To clearly define our scope, our project focuses strictly on fingerspelling and supports 24 static letters (A–Y, excluding J and Z, which depend heavily on motion trajectories rather than static hand posture).

The primary area where our project diverges from isolated-image classification is in video processing and temporal aggregation. Most publicly available ASL implementations focus on live webcam input with static bounding boxes. In contrast, our project processes pre-recorded video files and dynamically tracks the hand across the frame. Even when frame-level classification is highly accurate, converting a sequence of frame predictions into a correctly spelled word remains challenging. The system must ignore transitional frames, tolerate jitter, and decide when a letter has been "held" long enough to register.

Classical sequence models, such as Hidden Markov Models (HMMs), have long addressed recognition over time in related domains like speech [8]. More modern approaches, such as Connectionist Temporal Classification (CTC), address the labeling of unsegmented sequences directly and are conceptually relevant to our challenges [9]. This temporal issue is also deeply connected to broader skeleton-based action recognition research, which shows that while skeletal representations are highly informative, temporal modeling is often the deciding factor for robust recognition under motion and noise [10]. That perspective aligns with our findings: static per-frame classification is largely solved within our pipeline, while robust, continuous transcription remains the more difficult, unsolved component.

3. Experimental Platform

The experimental platform for this project evolved from a localized, file-based testing environment into a fully realized, real-time application. The system leverages a standard local development environment utilizing Python and broadly available computer vision libraries. No specialized or proprietary hardware is required, as the pipeline was designed to be highly accessible and capable of processing both pre-recorded MP4 video files for evaluation and standard live webcam feeds for real-time user interaction.

3.1. Software Stack and Technologies

The implementation relies on a robust, open-source software stack to handle everything from frame extraction to predictive inference:

- **Python:** Serves as the primary programming language, binding the computer vision and machine learning components together.
- **OpenCV:** Utilized for all core image processing tasks, including video capture, frame extraction, color-space conversion (BGR to RGB), and rendering the visual overlays for the user interface.

- **MediaPipe Hands:** Google's open-source framework provides the real-time hand tracking and landmark detection engine. MediaPipe identifies a 21-point three-dimensional skeletal representation of the hand, tracking the wrist and all individual finger joints regardless of the hand's position within the frame.
- **NumPy:** Handles the complex numerical operations required for feature engineering, coordinate normalization, and array manipulations.
- **scikit-learn:** Provides the machine learning backend for model training, data scaling, and live inference.
- **joblib:** Used for the efficient serialization and storage of trained model artifacts and data scalers.

3.2. System Pipeline and Modular Design

The implementation is organized into highly modular components to separate the concerns of landmark extraction, feature engineering, classification, and temporal aggregation. This modularity ensures the codebase remains maintainable and scalable. The core repository is structured as follows:

- **modules/landmark_extractor.py:** Manages both static-image and continuous video frame landmark extraction interfacing directly with MediaPipe Hands.
- **modules/feature_engineer.py:** Responsible for spatial normalization and feature computation, transforming the raw 21 hand landmarks into a standardized numerical format suitable for model ingestion.
- **modules/classifier.py:** Handles the loading of pre-trained models and scalers, executing the core inference logic on incoming feature vectors.
- **modules/temporal_aggregation.py:** Manages the complexities of time-series gesture recognition. This includes confidence filtering, sliding-window smoothing, segment detection, and managing the state of the live transcription buffer.
- **src/live_webcam.py:** Acts as the main execution script for the end-user webcam interface, integrating predictions, transcription, and visual overlays.
- **model_training/preprocess_data.ipynb & model_training/train_model.ipynb:** Jupyter notebooks dedicated to dataset preprocessing, automated feature extraction, classifier training, model comparison, evaluation, and the final export of training artifacts.

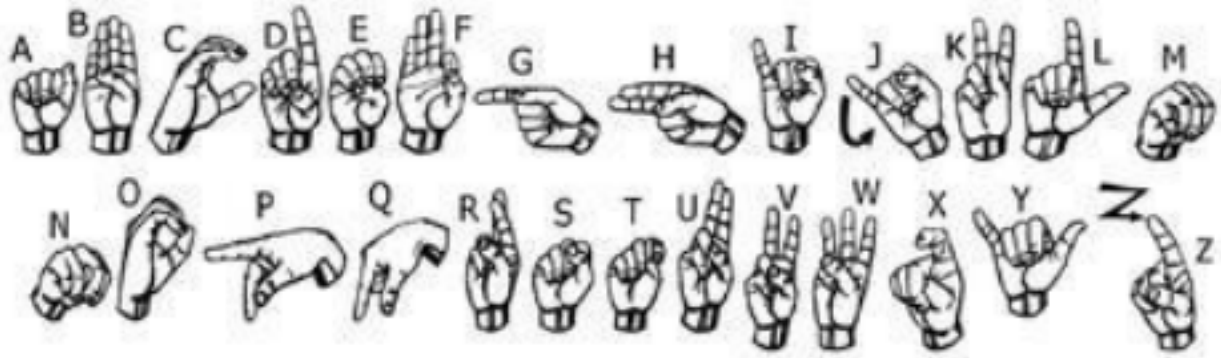


Figure 1: Visualization of the English ASL Alphabet (Image Credit: [11]).



Figure 2:

Example ASL fingerspelling for the English alphabet. The left image shows the gesture corresponding to the letter "F", and the right image shows the gesture corresponding to the letter "K" (Image Credit: [1]).

3.3. Data Acquisition and Evaluation Methodology

Translating static frame predictions into a fluid, readable transcription requires strict runtime configuration and temporal smoothing. The system is designed to digest both pre-recorded MP4 video files and standard live webcam input streams. For live webcam input, each frame is horizontally mirrored to provide an intuitive, usability-focused interaction for the user; for MP4 processing, the original frame orientation is strictly preserved. In both operational modes, frames are converted from BGR to RGB and passed into the MediaPipe tracking engine.

The system's configuration file dictates a stringent threshold to ensure accuracy:

- MediaPipe Detection Confidence: Set to 0.7 to ensure hands are only registered when clearly visible.
- MediaPipe Tracking Confidence: Set to 0.5 to maintain the skeletal lock during fast or obscured movements.

Once the extracted landmark coordinates are passed to the classification component for gesture recognition, the predicted letters are aggregated over time to form cohesive words. To mitigate rapid flickering and false positives, the `temporal_aggregation.py` module utilizes:

- A sliding window of 10 frames.
- A minimum stable segment length of 5 frames.
- A confidence threshold of 0.6.

These specific hyperparameter settings reflect the ultimate design goal of the system: aggressively suppressing short-lived visual noise and transitional frame errors while maintaining a highly responsive live interaction for the user.

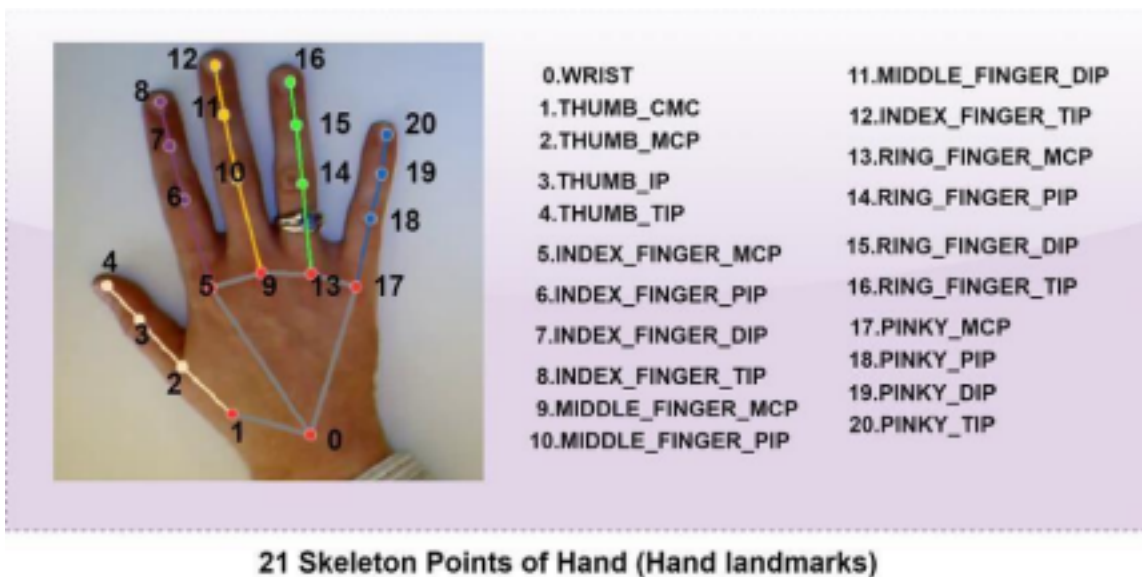


Figure 3: Visualization of the 21-point hand skeleton model for landmark-based gesture recognition (Image Credit: [12]).

4. Methodology

Our methodology follows a comprehensive, four-stage pipeline designed to bridge the gap between static image classification and dynamic gesture tracking: landmark extraction, feature engineering, supervised classification, and temporal aggregation.

4.1. System Overview and Dataflow

The proposed system is a robust ASL fingerspelling transcription pipeline that evolved to support both pre-recorded MP4 files and live video input. Each input video (or live stream) is first decomposed into individual frames using a standard frame extraction library such as OpenCV, ensuring a consistent frame rate and resolution for downstream processing.

For every frame, the system applies MediaPipe Hands to dynamically localize and track the signing hand anywhere in the image, avoiding the need for a fixed bounding box or constrained camera setup. The detector outputs a 21-point three-dimensional skeletal model of the hand, including the wrist and finger joints, which serves as a structured, low-dimensional representation of the gesture. These landmark coordinates are transformed into engineered features and passed to a classification module that predicts the most likely ASL alphabet letter per frame. The per-frame predictions are then temporally aggregated across the video to produce a final, smoothed letter or word-level transcription.

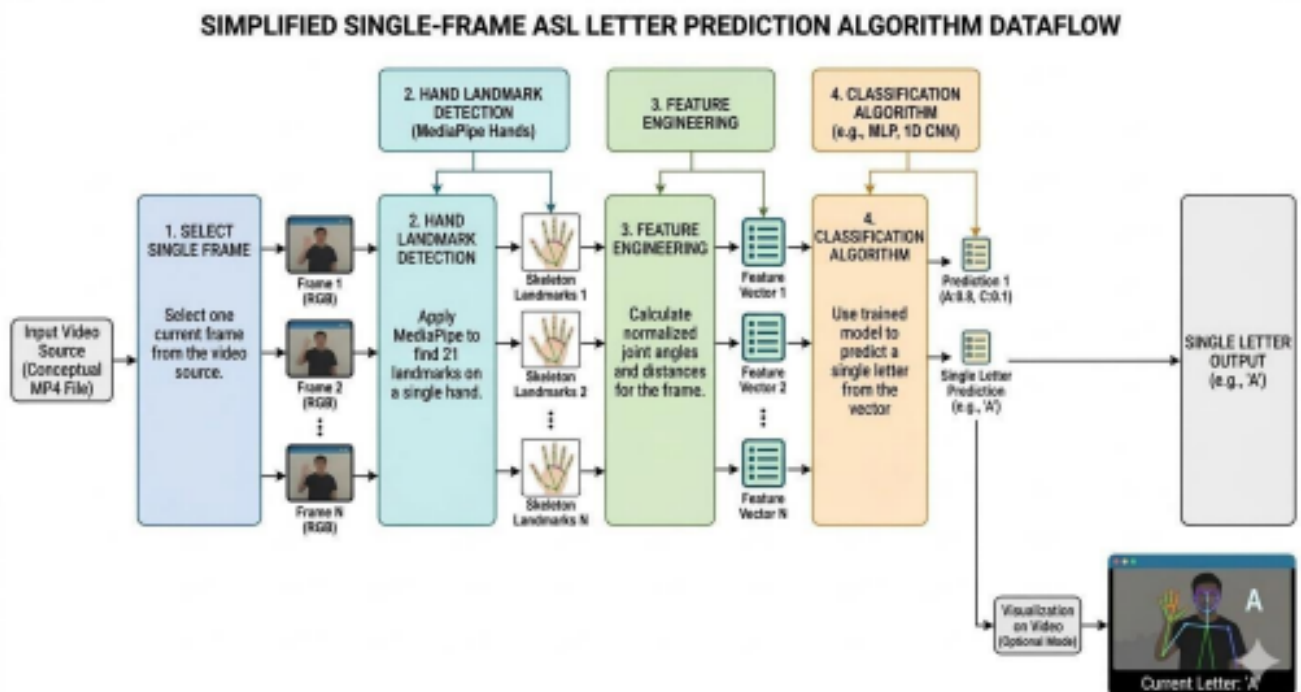


Figure 4: Visualization of algorithm. (Student Made)

4.2. Algorithms

4.2.1. Frame Extraction

- **Decoding:** Load the MP4 file or webcam feed and decode it into a sequence of RGB frames at a fixed frame rate (e.g., 15–30 fps) to balance temporal resolution and computation time.
- **Normalization:** Resize frames to a standard resolution and normalize the color space so that illumination and scale differences are reduced before hand detection occurs.

4.2.2. Hand Landmark Detection

- **Skeletal Estimation:** Apply the MediaPipe Hands model to each frame to detect the presence of hands and estimate a 21-point hand skeleton. Each landmark includes normalized x, y, and z coordinates.
- **Execution Modes:** During preprocessing on still training images, the detector runs in static-image mode. During live inference and video processing, it runs in tracking mode to improve computational speed and maintain temporal consistency.
- **Target Selection:** When multiple hands are detected in the frame, the implementation selects the dominant signing hand based on the highest detector confidence score or bounding-box size, and tracks that hand consistently across successive frames.
- **Missing Data:** If no confident hand detection is available, the frame is skipped during the training phase, or explicitly treated as a "no-hand event" (missing data) during live inference so that downstream modules can handle gaps in the sequence.

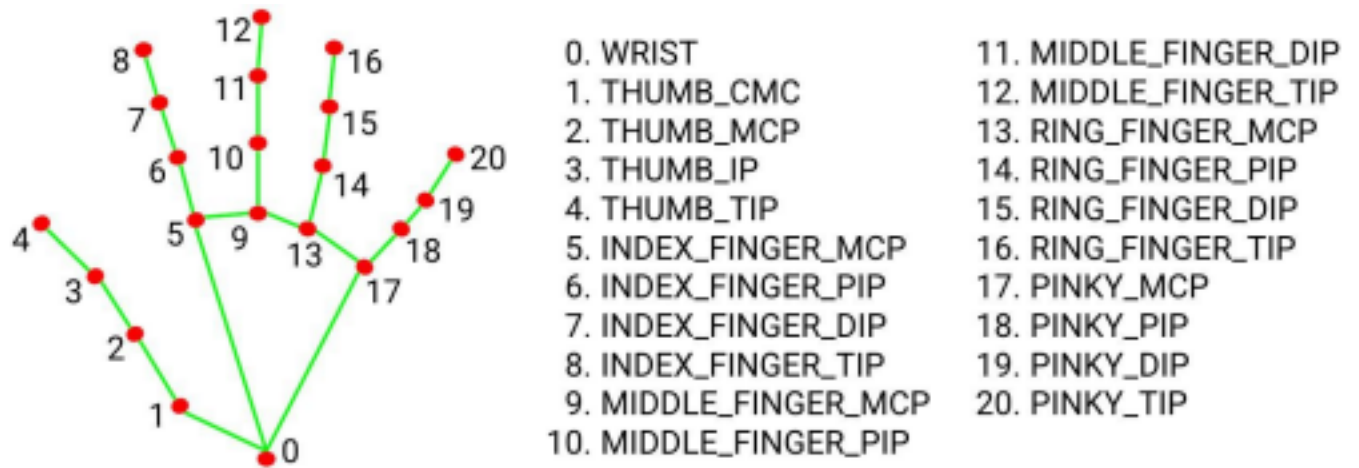


Figure 5: Visualization of the 21-point hand skeleton model for landmark-based gesture recognition (Image Credit: [13]).

4.2.3. Dataset Preparation and Feature Engineering

The system relies on accessible Kaggle datasets to train the models and increase success rates. Specifically, the training notebooks use the ASL Alphabet image dataset from Kaggle [7].

- **Class Restriction:** We restricted the dataset to the 24 supported static letters, purposefully excluding motion-based letters like J and Z, which require trajectory modeling rather than static posture classification.
- **Data Yield:** The preprocessing notebook evaluated 72,000 candidate images across those supported classes. After landmark extraction and filtering out images where MediaPipe failed to confidently detect a hand, the final training CSV used for model training contained 52,419 highly usable examples.
- **Structure:** The final dataset consisted of 89 columns total: the image path, the ground truth label, and the 87 engineered features. This train/test split is performed after landmark extraction and feature generation, ensuring that the reported classification results accurately reflect the usable subset for which MediaPipe successfully produced a hand skeleton.

Each detected hand is converted into a highly structured, 87-dimensional feature vector. We begin by normalizing the landmark coordinates to obtain a representation that is invariant to translation and scale. The hand is translated so that the wrist landmark becomes the origin point. We then normalize the overall scale using the Euclidean distance from the wrist to the middle-finger MCP joint. This makes the representation significantly more robust to the signer's distance from the camera and their overall hand size.

After normalization, the 87-dimensional feature vector is constructed from four core groups of higher-level geometric descriptors:

- **Flattened Landmark Coordinates:** All 21 landmarks retain their normalized x, y, and z coordinates, producing 63 distinct values.
- **Fingertip Distances:** We compute all pairwise Euclidean distances among the five fingertips, and then add the specific distances from the thumb tip to the other four fingertips.
- **PIP Joint Angles:** We compute an internal angle at each finger's PIP joint using the adjacent landmark vectors.
- **Finger Extension Ratios:** For each finger, we divide the straight-line MCP to-tip distance by the summed bone-segment length. This provides a compact, binary-like measure of how extended or curled each finger is to help distinguish visually similar letters.

These features were meticulously chosen to balance interpretability, compactness, and robustness. They encode global hand pose, local articulation, and relative finger relationships while remaining small enough for efficient classical classifiers.

4.2.4. Classification Algorithm and Training

- **Model Selection:** We trained supervised classifiers on the landmark-based feature vectors labeled with the ASL alphabet letters. While candidate models included MLPs and lightweight recurrent models, our final implementation directly compared two robust classical models: a Random Forest with 200 trees, and a Support Vector Machine (SVM) with an RBF kernel and probability estimates enabled.
- **Data Splitting:** String labels were converted to numeric class indices using a label encoder. The full feature matrix was split into training and test sets using an 80/20 stratified split with a fixed random seed of 42. This produced 41,935 training samples and 10,484 test samples.

- **Scaling:** Because the SVM is highly sensitive to feature scale, a StandardScaler was fit strictly on the training set and subsequently applied to both training and test data. The Random Forest classifier used the unscaled features directly.
- **Inference:** During inference, the trained model runs on each frame's feature vector to obtain a probability distribution over the letters, retaining both the predicted class and its confidence score.

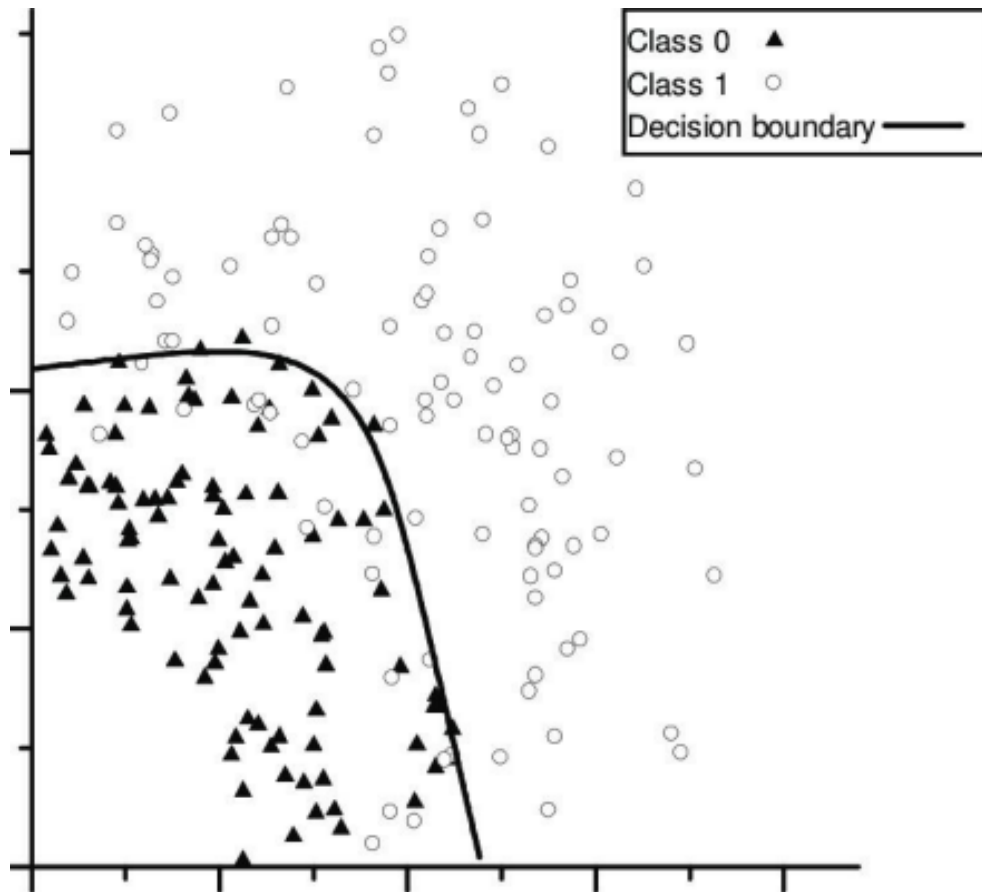


Figure 6: Visualization of the machine learning classification (Image Credit: [14]).

4.2.5. Temporal Aggregation

Per-frame predictions are not directly committed to text, as this would result in a highly erratic output. Instead, the live system stores recent predictions in a rolling window and performs temporal aggregation to smooth short-term jitter and noise.

The live transcriber manages this through a strict five-step heuristic process:

- **Reject:** Discard any per-frame predictions that fall below the established confidence threshold.
- **Vote:** Apply majority voting over the most recent rolling window of frame level predictions.
- **Smooth:** Estimate a smoothed confidence score derived exclusively from the frames that support the winning label.
- **Extend:** Start or extend the current temporal segment if the smoothed letter remains stable across consecutive frames.
- **Commit:** A letter is committed to the final transcription only when the current segment reaches the minimum frame-length threshold, and the same letter has not already been committed for the current hold.

This mechanism is simple but highly effective for isolated or deliberately held letters. It systematically ignores transitional frames between signs and prevents brief misclassifications from immediately appearing in the final transcript.

4.3. Data Structures

- **Frame Records:** Represent each frame as a comprehensive record containing: the frame index, timestamp, raw hand landmarks, the engineered 87-dimensional feature vector, and the classifier output probabilities.
- **Video Arrays:** Store an entire pre-recorded video as an ordered list (or array) of frame records, enabling efficient iteration for training, inference, and post-processing visualization.
- **Temporal Segments:** Maintain a separate structure for temporal segments, where each segment stores its start/end frame indices, the aggregated letter prediction, and its smoothed confidence. This supports word-level reconstruction and deeper error analysis.

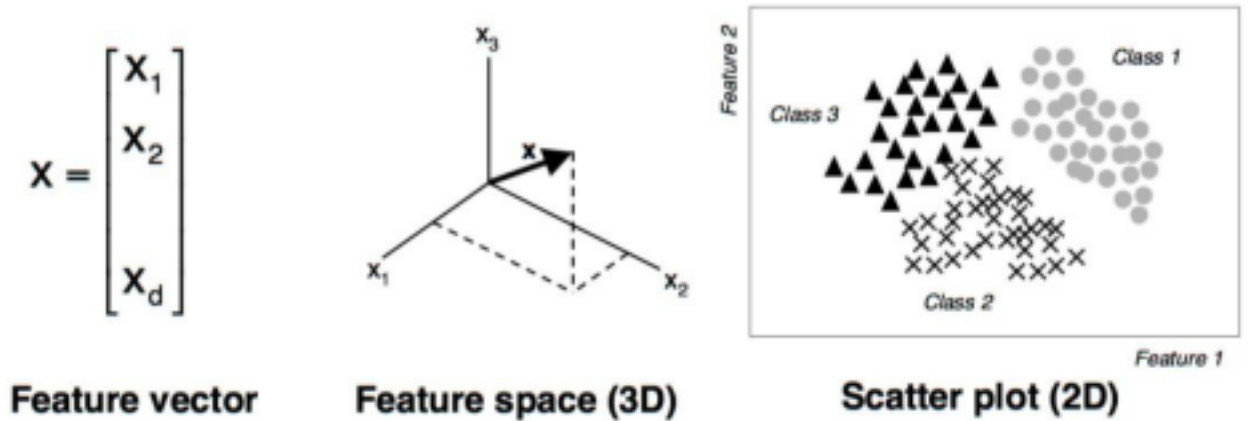


Figure 7: Visualization of feature vectors (Image Credit: [15]).

4.4. Graphical User Interface (GUI) and Control Room

To elevate the usability of the system beyond standard command-line execution and basic OpenCV display windows, a comprehensive Graphical User Interface (GUI) was developed. Built utilizing Python's tkinter and ttk libraries for native desktop rendering, alongside PIL (Pillow) for converting OpenCV image arrays into GUI-compatible formats, this lightweight application acts as a dynamic "control room" for the live transcription pipeline. The application can be launched via `src/simple_ui.py`, while the legacy OpenCV-only stream remains available for lower-overhead testing.

The interface is intentionally structured into a two-column layout to separate visual feedback from system operations:

4.5. Visual Feedback and Transcription Display

The primary left-hand column is dedicated to visual output. It houses a dynamically resizing video feed that displays the live webcam capture, horizontally mirrored for an intuitive user experience. The MediaPipe 21-point skeletal hand annotations are overlaid directly onto this feed in real-time. Immediately below the video feed is a dedicated, high-visibility "Transcription" banner, which displays the continuously appended text output generated by the temporal aggregation module.

4.6. Real-Time Metrics and Human-in-the-Loop Controls

The right-hand sidebar provides the user with comprehensive control over the system state and the transcription buffer. It is divided into three functional "cards":

System Controls: Instead of relying on obscure keyboard shortcuts bound to an OpenCV window, users are provided with clear, clickable buttons to start and stop the camera feed. Crucially, this panel includes human-in-the-loop editing tools such as Add Space, Backspace, and Clear Text. Because the core classifier handles static letters rather than continuous spatial boundaries, these buttons allow the user to manually separate words or correct minor misclassifications on the fly, making the tool significantly more practical for spelling full sentences.

Live Metrics: This read-out provides transparency into the classifier's immediate state. It displays the current raw letter prediction alongside its exact confidence percentage, as well as the overall system status (e.g., "Loading model...", "Running", "Camera stopped").

4.7. Dynamic Hyperparameter Tuning

The most significant technical advantage of the new GUI is the "Recognition Settings" panel. Previously, temporal aggregation hyperparameters were hardcoded into a configuration file, requiring a complete system restart to modify. The GUI exposes these variables through interactive sliders:

Confidence Threshold (0.10 - 0.95): Allows the user to dictate how strict the classifier must be before acknowledging a prediction.

Smoothing Window (3 - 20 frames): Adjusts the size of the rolling majority-vote buffer, enabling the user to balance rapid responsiveness against jitter reduction.

Min Segment Length (2 - 12 frames): Controls how long a gesture must be held before it is permanently committed to the transcription banner.

By binding these Tkinter scale widgets directly to the LiveTranscriber state, users can tune the system's sensitivity in real-time. This dynamic adjustment allows the software to be rapidly calibrated for different users, signing speeds, or changing lighting conditions without disrupting the live demonstration.

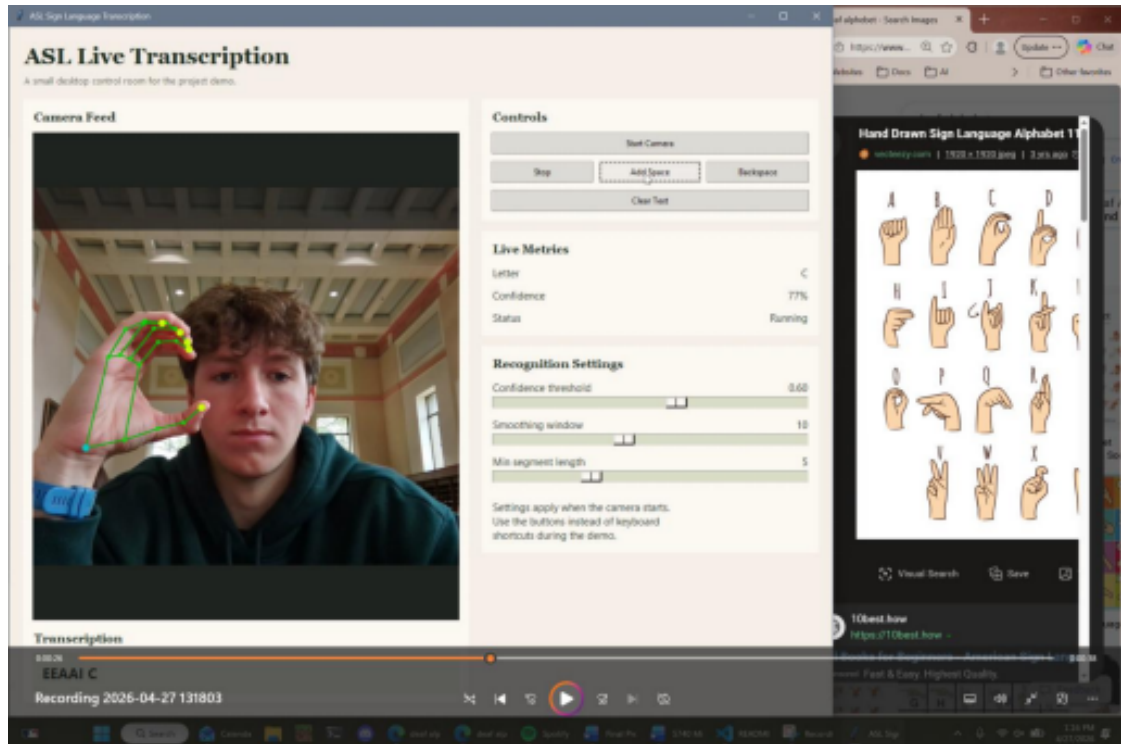


Figure 8: Using the UI

5. Results

We report results at two levels: quantitative isolated-letter classification and qualitative end-to-end live transcription behavior.

5.1. Landmark-Based Feature Dataset

The preprocessing pipeline successfully converted a large subset of the supported ASL alphabet dataset into geometric landmark features. From 72,000 candidate images across the 24 supported classes, 52,419 examples were retained for training and evaluation after MediaPipe extraction and filtering. This result is highly encouraging, as it demonstrates that off-the-shelf landmark detection is robust enough to bootstrap a large training set without requiring manual annotation of joint positions.

The feature representation also remained highly compact. Each sample is described by 87 numerical features instead of raw image pixels, which greatly reduced training complexity and facilitated the use of lightweight, classical machine learning models.

5.2. Classifier Comparison

Model	Test Accuracy
Random Forest (200 Trees)	99.27%
SVM (RBF kernel)	99.28%

Table 1. Classification accuracy on the 10,484-sample held-out test split.

The SVM achieved the best overall performance at 99.28%, narrowly outperforming the random forest baseline by 0.01 percentage points. Given this result, the SVM was selected as the final deployed model and serialized to `models/best_model.pkl` alongside its corresponding scaler and label encoder.

The detailed classification report indicates that performance was consistently high across almost all classes. The weakest performance was isolated to a small number of visually similar letters, specifically "M" and "N" because the distinction relies solely on the placement of the thumb on the fingers which can be occluded. This slight drop in relative accuracy aligns with the intuitive difficulty of distinguishing tightly folded finger configurations from static landmarks alone. Because the primary distinction between these letters relies on the exact placement of the thumb beneath the fingers—which is frequently obscured from a single 2D camera angle—the model occasionally struggles to differentiate them with the same confidence as more open, articulated gestures like "A" or "F".

5.3. End-to-End Live Webcam Transcription

To evaluate the pipeline in a real-world scenario, the system was deployed into a fully featured desktop application (`src/simple_ui.py`). In this interface, the system dynamically detects a single hand, predicts the current letter, visualizes the MediaPipe skeleton, and accumulates a smoothed transcription string over time.

Qualitatively, the live system is highly accurate when a signer clearly presents isolated static letters for several consecutive frames. To manage the inherent noise of live video, the UI provides real-time sliders for Confidence Threshold, Smoothing Window, and Minimum Segment Length. We observed that allowing the user to dynamically tune these hyperparameters was critical for success; increasing the smoothing window effectively suppressed noisy fluctuations, while adjusting the segment length accommodated different signing speeds. The inclusion of human-in-the-loop editing controls (Space, Backspace, Clear) further proved the system's viability as an interactive, real-time prototype.

However, the system remains fragile when letters are signed too rapidly, when transition frames between signs dominate the sequence, or when the hand temporarily leaves the camera view. These observations confirm the hypothesis raised during peer review: static frame classification is largely a solved problem within this domain, but robust temporal segmentation across continuous, fluid signing remains a significant ongoing challenge. At the time of this report, we present the live webcam application as a highly functional end-to-end demonstration rather than a benchmarked continuous signing solution.

5.4. Pre-Recorded MP4 Processing

In addition to live webcam inference, we successfully validated the core objective of our original proposal: processing pre-recorded video files. The system ingests standard MP4 files, extracts them frame-by-frame, and applies the exact same landmark extraction, feature engineering, and temporal aggregation pipeline used in the live environment.

By decoupling the frame extraction rate from the constraints of live hardware rendering, the MP4 processing pipeline provides a reliable method for bulk-analyzing existing videos. The output generates an annotated video file featuring the overlaid MediaPipe skeleton and an aggregated text transcript.

This confirms the viability of our "video-first architecture" for target users like Deaf content creators and archivists who need asynchronous captioning without being tethered to a live, bounded-box webcam feed.

6. Discussion

6.1 Revisiting the Core Bottleneck: Temporal Segmentation

One of the most consistent themes across both implementation and peer feedback is that temporal segmentation and not classification is the central challenge of this problem. While our classifier achieves near-perfect accuracy on isolated frames, this performance does not directly translate to reliable transcription in continuous video.

This gap arises because real signing is not composed of clean, discrete frames corresponding to letters. Instead, it consists of continuous motion with ambiguous transition states. Between two letters, there may be multiple intermediate hand configurations that do not correspond to any valid class but are still processed by the classifier. As noted in peer feedback, a naïve majority-voting scheme can incorrectly merge adjacent letters or suppress valid ones if segmentation boundaries are misaligned.

Our current heuristic approach (confidence filtering, sliding window voting, and minimum segment thresholds) works well when letters are deliberately held. However, it struggles in three key scenarios:

- Rapid signing, where stable segments are too short to meet threshold requirements
- Transition-heavy sequences, where intermediate frames dominate
- Ambiguous boundaries, where two letters blend into one continuous motion

These limitations highlight an important insight: temporal aggregation is not just a smoothing problem, but a segmentation problem. Future improvements must explicitly model transitions rather than treating them as noise.

6.2 Domain Gap: Static Images vs. Real Video

Another critical issue raised in peer review is the mismatch between training data and deployment conditions. Our model is trained on clean, static images, while inference occurs on dynamic video streams. This domain gap introduces several sources of error:

- Motion blur reduces landmark precision
- Partial occlusion (self-occlusion or frame boundaries) disrupts feature consistency
- Viewpoint variation alters perceived geometry
- Lighting changes affect detection confidence

Even though MediaPipe is robust, it is not invariant to all real-world conditions. As a result, some frames in video sequences produce noisy or incomplete landmark estimates, which propagate through the pipeline. This explains why a model achieving 99% accuracy on static data can still produce unstable predictions in practice. The issue is not model capacity, but data mismatch. Bridging this gap would likely require:

- Training on video-derived landmark sequences rather than static images
- Introducing data augmentation (e.g., noise, jitter, occlusion simulation)
- Incorporating temporal consistency constraints during training

6.3 Failure Cases and Error Modes

To better understand system limitations, it is useful to categorize common failure modes observed during testing:

1. **Visually Similar Letters:** Letters such as M and N remain difficult to distinguish due to subtle differences in thumb placement. These differences are often occluded or poorly captured in a 2D projection.
2. **Fast Transitions:** When users sign quickly, the system may skip letters entirely because the segment does not persist long enough to be committed.
3. **Repeated Letters:** The system sometimes fails to distinguish between intentional repetition (e.g., “LL”) and a prolonged hold of a single letter.
4. **Tracking Loss:** Temporary loss of hand detection (e.g., moving out of frame) introduces gaps that disrupt temporal continuity.
5. **Multi-Hand or Background Interference:** Although the system selects a dominant hand, additional hands or complex backgrounds can occasionally confuse detection.

These failure modes reinforce that the problem extends beyond classification accuracy and into robust perception under real-world variability.

7. Conclusion and Future Work

This project presented a complete pipeline for ASL fingerspelling transcription based on hand skeleton tracking and classical machine learning. By leveraging MediaPipe for 21-point landmark extraction and combining it with normalized geometric features, the system achieves highly accurate recognition of static ASL letters. The best-performing model, an RBF-kernel SVM, reached 99.28% accuracy on a held-out test set, demonstrating that structured landmark representations are sufficient for robust isolated-letter classification.

Beyond classification, the project successfully integrates perception, feature engineering, inference, and temporal aggregation into a unified system capable of processing both live webcam input and pre-recorded video. This end-to-end design validates the feasibility of a “video-first” approach to ASL transcription, addressing a key limitation of prior systems that rely on constrained webcam setups.

At the same time, the work clearly demonstrates that continuous transcription remains an open challenge. While frame-level predictions are highly reliable, converting them into accurate, stable word sequences requires more advanced temporal modeling. The current heuristic-based aggregation approach provides a functional baseline but does not fully resolve issues related to transition frames, variable signing speed, and motion-based gestures.

As a result, the system should be viewed as a strong prototype of a recognition backbone rather than a complete transcription solution. It establishes that the perceptual front-end hand tracking and geometric representation is largely solved within this context, while highlighting temporal segmentation as the primary area for future research.

Future work should focus on integrating dynamic motion tracking to support the letters 'J' and 'Z', as well as improving sequence modeling and bridging the gap between static training data and real-world video input. Promising directions include velocity-based segmentation, explicit transition-state modeling, and end-to-end sequence learning approaches such as CTC or transformer-based architectures.

Ultimately, this project demonstrates that accurate ASL fingerspelling recognition is achievable with lightweight, interpretable methods, and that the next step toward practical deployment lies in robust temporal understanding of continuous signing.

REFERENCES

- [1] A. Nagaraj. ASL Alphabet dataset. Kaggle. https://www.kaggle.com/datasets/grassknoted/asl_alphabet (accessed April 27, 2026).
- [2] F. Zhang, V. Bazarevsky, A. Vakunov, A. Tkachenka, G. Sung, C.-L. Chang, and M. Grundmann. "MediaPipe Hands: On-device real-time hand tracking." arXiv preprint arXiv:2006.10214, 2020. <https://arxiv.org/abs/2006.10214>
- [3] H. Kolivand, S. Joudaki, M. S. Sunar, M. M. Arabnia, and D. Tully. "A new framework for sign language alphabet hand posture recognition using geometrical features through artificial neural network (part 1)." Neural Computing and Applications, 33, 4945-4963, 2021. <https://doi.org/10.1007/s00521-020-05279-7>
- [4] V. S. Ch, H. B. Gogineni, B. Prasad, and P. Praveen Kumar. "Real-Time Recognition of American Sign Language Using Mediapipe and Machine Learning Techniques." Journal of Information Systems Engineering and Management, 10(3), 2025. <https://doi.org/10.52783/jisem.v10i3.9436>
- [5] M. Huq, S. Bhuyan, and S. Islam. "American Sign Language Recognition for Alphabets Using MediaPipe and LSTM." Procedia Computer Science, 215, 642-651, 2022. <https://doi.org/10.1016/j.procs.2022.12.066>
- [6] P. Molchanov, X. Yang, S. Gupta, K. Kim, S. Tyree, and J. Kautz. "Online Detection and Classification of Dynamic Hand Gestures with Recurrent 3D Convolutional Neural Networks." Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 2016.
- [7] N. C. Camgoz, O. Koller, S. Hadfield, and R. Bowden. "Neural Sign Language Translation." Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 2018.
- [8] L. R. Rabiner. "A tutorial on hidden Markov models and selected applications in speech recognition." Proceedings of the IEEE, 77(2), 257-286, 1989 <https://doi.org/10.1109/5.18626>
- [9] A. Graves, S. Fernandez, F. Gomez, and J. Schmidhuber. "Connectionist temporal classification: Labelling unsegmented sequence data with recurrent neural networks." In Proceedings of the 23rd International Conference on Machine Learning, pp. 369-376, 2006. <https://doi.org/10.1145/1143844.1143891>
- [10] H. Duan, Y. Zhao, K. Chen, D. Lin, and B. Dai. "Revisiting skeleton-based action recognition." arXiv preprint arXiv:2104.13586, 2021. <https://arxiv.org/abs/2104.13586>

[11] Hearview.ai. "The Ultimate Guide to Learning the ASL Alphabet." [Online]. Available: <https://www.hearview.ai/blogs/news/the-ultimate-guide-to-learning-the-aslalphabet>

[12] ResearchGate. "21 Skeleton points of hand (Hand landmarks)." [Online]. Available: https://www.researchgate.net/figure/21-Skeleton-points-of-handHand-landmarks_fig4_379653767

[13] Google AI Edge. "MediaPipe Hands." GitHub. [Online]. Available: <https://github.com/google-aiedge/mediapipe/blob/master/docs/solutions/hands.md>

[14] ResearchGate. "The decision boundary is shown for a neural network classifier with two hidden units." [Online]. Available: https://www.researchgate.net/figure/The-decision-boundary-is-shown-for-a-neural-net-workclassifier-with-two-hidden-units_fig4_13937960

[15] Brilliant.org. "Feature Vector." [Online]. Available: <https://brilliant.org/wiki/feature-vector/>

APPENDIX B: Source Code

This appendix contains the core algorithms developed for spatial feature extraction and real-time temporal smoothing.

1. Spatial Feature Extraction

The following function from `feature_engineer.py` takes raw normalized hand landmarks and computes a 1D feature vector consisting of coordinates, pairwise fingertip distances, PIP joint angles, and finger extension ratios.


```

9
10 def compute_features(landmarks):
11     # Normalize position (center at wrist) and scale
12     normalized_landmarks = normalize_landmarks(landmarks)
13
14     # Flatten coordinates to a 1D vector
15     coordinate_features = flatten_coordinates(normalized_landmarks)
16
17     # Compute pairwise distances between fingertips
18     fingertip_distance_features = compute_fingertip_distances(normalized_landmarks)
19
20     # Compute angles at the PIP (middle) joint of each finger
21     pip_angle_features = compute_pip_angles(normalized_landmarks)
22
23     # Compute finger extension ratios (how straight vs curled each finger is)
24     extension_ratio_features = compute_extension_ratios(normalized_landmarks)
25
26     # Concatenate all feature groups into one flat vector
27     feature_vector = np.concatenate([
28         coordinate_features,
29         fingertip_distance_features,
30         pip_angle_features,
31         extension_ratio_features
32     ])
33
34     return feature_vector.astype(np.float32)
35

```

2. Real-Time Temporal Aggregation

The LiveTranscriber class from temporal_aggregation.py manages a rolling window of recent predictions. It uses majority voting and confidence thresholds to stabilize the output and construct a reliable transcription string from erratic frame-to-frame classifications.

```

151 class LiveTranscriber:
152     def __init__(self, window_size = None, min_segment_length = None, confidence_threshold = None):
153         self.window_size = window_size or config.WINDOW_SIZE
154         self.min_segment_length = min_segment_length or config.MIN_SEGMENT_LENGTH
155         self.confidence_threshold = confidence_threshold or config.CONFIDENCE_THRESHOLD
156
157         # Rolling window of (letter, confidence) for the recent frames
158         self.recent_predictions = deque(maxlen=self.window_size)
159
160         # Current stable segment tracking
161         self.current_segment_letter = None
162         self.current_segment_length = 0
163         self.last_committed_letter = None
164
165         # The growing transcription string
166         self.transcription = ""
167
168         # What to display on-screen right now (smoothed, not necessarily committed)
169         self.display_letter = None
170         self.display_confidence = 0.0
171
172     def update(self, letter, confidence):
173         if confidence < self.confidence_threshold:
174             self.display_letter = None
175             self.display_confidence = 0.0
176             self._reset_segment()
177             return
178
179         self.recent_predictions.append((letter, confidence))
180
181         # Majority vote over the rolling window
182         window_letters = [p[0] for p in self.recent_predictions]
183         letter_counts = Counter(window_letters)
184         smoothed_letter, _ = letter_counts.most_common(1)[0]
185
186         # Average confidence of frames that voted for the winner
187         winner_confidences = [p[1] for p in self.recent_predictions if p[0] == smoothed_letter]
188         smoothed_confidence = sum(winner_confidences) / len(winner_confidences)
189
190         self.display_letter = smoothed_letter
191         self.display_confidence = smoothed_confidence
192
193         # Extend or start a new segment
194         if smoothed_letter == self.current_segment_letter:
195             self.current_segment_length += 1
196         else:
197             self.current_segment_letter = smoothed_letter
198             self.current_segment_length = 1
199             if self.last_committed_letter != smoothed_letter:
200                 self.last_committed_letter = None
201
202         # Commit when the segment first reaches the minimum length
203         if (self.current_segment_length == self.min_segment_length and
204             smoothed_letter != self.last_committed_letter):
205             self.transcription += smoothed_letter
206             self.last_committed_letter = smoothed_letter
207

```

APPENDIX C: Additional Resources

Below are external links related to the project, including a demonstration of the live transcription system and the full codebase.

Project Repository: The complete source code, including the model architecture, data collection scripts, and configuration files, can be found on our GitLab repository:

- [miles-nichols/ASL-Captioning-from-Video-Using-Hand-Skeleton-Tracking](#)
- Live Demonstration Video: A video demonstrating the real-time capabilities of the LiveTranscriber processing gesture inputs into text is available on YouTube: <https://www.youtube.com/watch?v=p3wNRFaX3kI>
- Dataset/Frameworks Used: * MediaPipe Hand Landmarks functionality used for initial spatial mapping:
https://developers.google.com/mediapipe/solutions/vision/hand_landmarker